

# Ce que le cours a enseigné, et ce qui en a été fait

Voici les leçons de la vidéo de formation (Claude Code Masterclass) que j'ai réellement collées dans le chat, entre le 4 et le 12 août 2026. Classées selon la vraie table des matières du cours, pas selon la date où je les ai collées.

## Méthode

J'ai analysé 8 sessions Claude Code ( `bef46bcb` , `bc2538f9` , `a056c64e` , `8d281e45` , `3bb6cf8f` , `6b055b65` , `62cf38a7` , `ecc9fb7c` ). Toutes les images collées dans le chat ont été extraites. Regardées une par une. Triées entre captures réelles de la vidéo du formateur et captures perso de debug (Firebase, terminal, Figma), ces dernières écartées de ces fichiers. Chaque leçon retenue est reliée au commit git qui l'a appliquée dans le code de Pocket Heist.

## Sommaire

| Section                                      | Contenu retrouvé                                                                                                                                               |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Section 1 : Introduction                     | Aucun. Les captures commencent après cette section                                                                                                             |
| Section 2 : Commands, Context, Tools & Hooks | Hooks <code>PostToolUse</code> , <code>log jq</code> , chaînage avec <code>prettier</code> , commandes <code>/commit-message</code> et <code>/component</code> |
| Section 3 : Plan Mode & Specs                |                                                                                                                                                                |

| Section                      | Contenu retrouvé                                                                                                                                                                           |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              | Template de spec, méthode de rédaction complète, prompt <code>/spec</code> , du spec au plan d'implémentation, plus la chronologie complète des 9 specs du projet ( <code>_specs/</code> ) |
| Section 4 : MCP Servers      | Plan Firebase MCP (Firestore + Auth). Figma MCP et Context7 sont utilisés mais aucune capture ne vient de la vidéo pour eux                                                                |
| Section 5 : Plugins & Skills | Aucun. Rien de capté ne correspond à ce thème                                                                                                                                              |

## À lire avec ces nuances

Deux sections (1 et 5) n'ont aucune leçon confirmée. Ce n'est pas un oubli, aucune capture des 8 sessions ne s'y rattache. Je préfère le dire plutôt que forcer un contenu.

Certaines leçons chevauchent plusieurs sections. La commande `/spec` est une «Command», mais son contenu enseigne la méthode de «Specs». Le chevauchement est signalé dans le fichier concerné, pas dupliqué.

Chaque fichier cite les commits git exacts (hash court, message, date) qui ont appliqué la leçon dans le projet. Vérifiables avec `git show <hash>`.

# Section 1 : Introduction

---

*Cours : Claude Code Masterclass, Section 1 (8/8 leçons, 39 min).*

Après lecture des 8 rapports de session ( `bef46bcb` , `bc2538f9` , `a056c64e` , `8d281e45` , `3bb6cf8f` , `6b055b65` , `62cf38a7` , `ecc9fb7c` ), aucune capture collée dans le chat ne correspond à la Section 1 du cours Udemy.

## Pourquoi rien n'a été rattaché ici

Toutes les captures identifiées dans les 8 sessions couvrent des sujets qui relèvent clairement des sections suivantes.

- **Section 2 (Commands, Context, Tools & Hooks)** : configuration du hook `PostToolUse` via `/hooks` (sessions `bc2538f9` et `a056c64e` ).
- **Section 3 (Plan Mode & Specs)** : template de spec, specs complètes du formateur (login/signup, `useUser` , logout, login form, route protection, create heist form) dans quasiment toutes les sessions à partir du 6 août.
- **Section 4 (MCP Servers)** : Firebase MCP (plan Firestore + Auth) dans `8d281e45` et `3bb6cf8f` .
- **Section 5 (Plugins & Skills)** : rien de confirmé non plus (voir ce fichier dédié).

Un seul indice indirect existe. Dans la session `bef46bcb` (04/08), le rapport mentionne un `/init` juste avant les premières images collées. Mais aucune capture n'illustre cette étape. Les 3 images de cette session montrent en réalité toutes le même mock de «skeleton loader », sans rapport avec une introduction au cours ou à l'outil.

De même, l'image de la session `8d281e45` qui montre l'interface Udemy (barre de progression du cours) affiche «Section 2 : 13/13 », «Section 3 : 6/6 », «Section 4 : 6/13 ». Pas la Section 1. Elle sert uniquement de repère chronologique pour un récapitulatif demandé, pas de contenu pédagogique de la Section 1.

## Conclusion

Pas de leçon à documenter pour la Section 1. Les 8 sessions capturées commencent après cette section, la première capture exploitable démarre directement dans le vif du sujet (hooks, Section 2). Aucun commit du projet n'est donc à rattacher ici.

## Section 2 : Commands, Context, Tools & Hooks

---

Cours : Claude Code Masterclass, Section 2 (13/13 leçons, 1h33).

Cette section couvre les hooks Claude Code (via `/hooks`) et les commandes personnalisées du projet. Deux sessions collent précisément à ce thème, `bc2538f9` (05/08) et `a056c64e` (06/08), toutes deux confirmées par un watermark Udemty ou un chemin `/Users/shaun/...` (la machine du formateur) visible sur les captures.

### Créer un hook `PostToolUse` pas à pas via `/hooks`

Sessions source : `bc2538f9` (05/08) et `a056c64e` (06/08)

**Ce que montre la vidéo :** l'écran interactif `/hooks` en 4 étapes. D'abord le choix de l'événement `PostToolUse`. Puis `Add new matcher...` avec la liste des `tool_name` valides (`Task`, `Bash`, `Read`, `Edit`, `Write`, etc.) et la syntaxe (`Write` seul, `Write|Edit` pour plusieurs outils, `Web.*` en regex). Puis retour à `/hooks` titré «Matcher: Write|Edit ». Puis `Add new hook...` avec le champ `Command:` rempli avec `echo "hello"`.

**Ce que le formateur explique :** un hook se configure en deux temps. D'abord un matcher, quel outil déclenche le hook. Puis une commande shell exécutée après. L'entrée reçue par la commande est un objet JSON (`tool_input`, `tool_response`). Le code de sortie détermine l'affichage, 0 veut dire stdout visible directement dans le transcript, sans besoin de `Ctrl+0`. Une capture vidéo confirme ce point en montrant le résultat `PostToolUse:Edit hook succeeded: hello` juste sous un diff.

**Application dans Pocket Heist :** le menu interactif `/hooks` ne s'est pas comporté comme prévu dans l'environnement WSL. J'ai obtenu le même résultat en éditant directement `.claude/settings.local.json` avec l'équivalent JSON (`"matcher": "Write|Edit"`, `"command": "echo \"hello\""`). Cette compréhension a ensuite été rédigée dans `.claude/lecons/hooks-guide.md` et `.claude/lecons/hooks-plan.md`, commit `19bfc5a` («docs: ajoute les leçons hooks et traduit les commandes en français », 06/08/2026).

### Logger l'objet complet transmis au hook avec `jq`

Session source : `a056c64e` (06/08)

**Ce que montre la vidéo :** le contenu d'un fichier `tool-use.json` produit par un hook. L'objet complet reçu, `session_id`, `cwd`, `hook_event_name`, `tool_name`, `tool_input`, `tool_response` avec `structuredPatch`, sur un fichier du projet du formateur (`app/(public)/login/page.tsx`, chemin `/Users/shaun/Code/Courses/pocket_heist`).

**Ce que le formateur explique :** pour logger l'objet JSON complet reçu par le hook, le bon filtre `jq` est l'identité `jq '.'` (garde tout). Pas un filtre restrictif comme `.tool_input.file_path` qui ne garderait qu'un seul champ.

**Application dans Pocket Heist :** correction de la commande dans `.claude/settings.local.json` (`jq . > tool-use.json`), mise à jour de `.claude/lecons/hooks-jq-log.md`, suppression d'un fichier obsolète (`tool-log.json`). Tout regroupé dans le commit `19bfc5a`, qui ajoute justement ce fichier `hooks-jq-log.md`.

## Châîner plusieurs commandes sur un même hook (permissions + auto-formatage `prettier`)

**Session source :** `a056c64e` (06/08)

**Ce que montre la vidéo :** le fichier `.claude/settings.local.json` du formateur avec des permissions pré-accordées (`WebSearch`, `Bash(git init)`, `Bash(git switch:*)`) et trois commandes chaînées sur le même matcher `Write|Edit`, `echo "hello"`, `jq . > tool-use.json`, et une commande de reformatage automatique des fichiers `.tsx` via `prettier`.

**Ce que le formateur explique :** un même matcher peut déclencher plusieurs hooks à la suite. Ici un exemple concret d'auto-formatage post-édition, qui filtre sur l'extension `.tsx` avant d'appeler `npx prettier --write`.

**Application dans Pocket Heist :** j'ai reproduit ces commandes dans `.claude/settings.local.json`, plus l'ajout d'un fichier `.prettierrc` (`semi: false`) pour que le hook `prettier` respecte la convention du projet, pas de points-virgules. Le tout dans le commit `19bfc5a` (06/08/2026).

## Commande personnalisée `/commit-message`

**Origine :** fournie dans le squelette de départ du cours (`.claude/commands/commit-message.md`), commit `10dd3cf` («feat: ajoute le squelette de départ du projet Pocket Heist », 06/08/2026), puis traduite en français dans le commit `19bfc5a` (même date).

**Ce qu'elle fait :** lance `git status` et `git diff --staged`, analyse les changements stagés, puis propose un message de commit au format `<emoji> <type>: <description>` (liste fixe d'emojis, `feat`, `fix`, `refactor`, `docs`, `style`, `test`, `perf`). Elle ne committe jamais automatiquement, seulement sur confirmation explicite.

*Aucune capture vidéo retrouvée dans les 8 sessions ne montre l'écran de création de cette commande. Sa présence est déduite du diff git et du récapitulatif de la session `8d281e45`, qui la cite comme exemple d'usage classé en Section 2.*

## Commande personnalisée `/component` (TDD)

**Origine :** ajoutée en français dans le commit `19bfc5a` (06/08/2026).

**Ce qu'elle fait :** à partir d'une description libre ( `$ARGUMENTS` ), elle détermine un nom de composant en PascalCase, écrit d'abord les tests ( `tests/components/[Nom].test.tsx` ), les lance (échec attendu), crée le composant ( `.tsx` + `.module.css` + `index.ts` ) selon la convention du projet, relance les tests (succès attendu), puis l'ajoute à `app/(public)/preview/page.tsx`. Un cycle TDD complet piloté par une seule commande slash.

*Même limite que `/commit-message`, pas de capture vidéo isolée retrouvée pour cette commande. Elle est citée dans le récapitulatif de session `8d281e45` comme faisant partie du travail classé en Section 2.*

## Chevauchement signalé : la commande `/spec`

La commande `/spec` (créée en `d4c3bc8`, «feat: ajoute la commande `/spec` et son template de spécification », 06/08/2026 17:00) pourrait sembler relever de cette Section 2, une commande personnalisée comme `/commit-message` ou `/component`. Je la classe en Section 3 «Plan Mode & Specs », parce que tout le contenu pédagogique retrouvé à son sujet (structure du template, contenu détaillé d'une spec complète, formulation du prompt d'entrée) enseigne la méthode de rédaction d'une spec, pas la mécanique de création d'une commande slash. Ça correspond à l'intitulé de la Section 3, pas de la Section 2. Voir `section-3-plan-mode-specs.md`.

---

**Note de couverture :** sur les 8 sessions analysées, seules `bc2538f9` et `a056c64e` contiennent des leçons vidéo directement liées aux hooks. Aucune session ne montre d'autres sujets typiques de cette section (contexte, autres outils du terminal). Soit ce contenu n'a pas été capturé, soit il correspond aux leçons de la Section 2 non couvertes par une capture d'écran.

## Section 3 : Plan Mode & Specs

---

*Cours : Claude Code Masterclass, Section 3 (6/6 leçons, 39 min).*

Cette section documente comment le formateur enseigne la méthode de rédaction de spec (via la commande `/spec` et un template fixe), puis comment il détaille cette spec en plan d'implémentation avant tout code. Les captures clés viennent des sessions `a056c64e` (structure canonique du template, plus un exemple complet) et `3bb6cf8f` (comment formuler le prompt qui lance `/spec`, puis deux exemples complets de spec et de plan).

### Le template de spec canonique

**Session source :** `a056c64e`, 06/08/2026

**Ce que montre la vidéo :** le contenu exact de `_specs/template.md` du formateur. Entête `# Spec for <feature-name>, branch:, figma_component (if used):`, puis les sections `## Summary`, `## Functional Requirements`, `## Figma Design Reference (only if referenced)`, `## Possible Edge Cases`, `## Acceptance Criteria`, `## Open Questions`, `## Testing Guidelines`.

**Ce que le formateur explique :** une spec suit toujours la même structure fixe, plus simple que ce que Claude avait improvisé au départ. La section Figma est explicitement conditionnelle («only if referenced»), à omettre si aucun design Figma n'est utilisé.

**Application dans Pocket Heist :** réécriture de `_specs/template.md` pour coller exactement à cette structure, commit `d4c3bc8` («feat: ajoute la commande `/spec` et son template de spécification», 06/08/2026). La commande `/spec` elle-même (créée dans ce même commit) relève plutôt de la [Section 2 «Commands»](#). Ici je ne retiens que le template de sortie qu'elle produit.

### Une spec complète et exploitable, exemple login/signup

**Session source :** `a056c64e`, 06/08/2026

**Ce que montre la vidéo :** la spec entièrement remplie `authentication-forms.md` du formateur pour les formulaires de connexion et d'inscription. Un Summary concret, 9 Functional Requirements précis (validation HTML5, accessibilité ARIA, responsive...), 8 Edge Cases, 8 Acceptance Criteria, et surtout 5 Open Questions déjà tranchées par le formateur lui-même. Pas de longueur minimale de mot de passe, pas de persistance, pas de «remember me», pas de «mot de passe oublié».

**Ce que le formateur explique :** à quoi ressemble une spec réellement exploitable, pas un simple squelette. Trancher les questions ouvertes soi-même dans le document évite les allers-retours pendant le plan et l'implémentation.

**Application dans Pocket Heist :** réécriture complète de `_specs/login-signup-forms.md` pour intégrer les manques repérés (accessibilité, responsive, touche Entrée, absence de section Figma car non utilisée), commit `df83e8d` («feat: ajoute les formulaires d'authentification login/signup », 10/08/2026, qui embarque spec, plan et code dans le même commit).

## Formuler le prompt qui lance `/spec`

**Session source :** 3bb6cf8f, 10/08/2026

**Ce que montre la vidéo :** le texte exact tapé par le formateur pour démarrer une spec. *«/spec let's spec an auth state management solution for the app, where we can access the current user (null if logged out, the user object if logged in), through a hook called useUser. [...] Do not spec any signup/login/logout flow yet, just a realtime global listener to update user status. »*

**Ce que le formateur explique :** bien formuler l'idée de fonctionnalité avant `/spec`, en nommant le résultat attendu (un hook `useUser`, son comportement) et en posant une limite claire («pas de flux login/logout ») pour empêcher la spec de dériver sur un périmètre trop large.

**Application dans Pocket Heist :** cette formulation a servi de référence directe pour cadrer le prompt `/spec auth-state-hook`, à l'origine de la branche `claude/feature/auth-state-hook` et du fichier `_specs/auth-state-hook.md`, commit `96f5001` («docs: spec + plan pour le hook useUser (état d'auth global) », 11/08/2026, 12:51).

## À quoi ressemble une spec complète, exemple useUser

**Session source :** 3bb6cf8f, 10/08/2026

**Ce que montre la vidéo :** le fichier `auth-state-management.md` du formateur, complet. Summary, Functional Requirements, Possible Edge Cases, Acceptance Criteria, Open Questions, Testing Guidelines pour le hook `useUser` (lecture seule de l'utilisateur courant, écoute temps réel via `onAuthStateChanged`, persistance après rafraîchissement, sans flux login/logout).



**Ce que le formateur explique :** une spec complète couvre tous les cas particuliers et critères d'acceptation, même pour une fonctionnalité en apparence simple. Point pédagogique clé, les Open Questions ne sont pas forcément résolues par écrit. Elles peuvent être discutées à l'oral pendant le mode plan ( /plan ), juste avant l'implémentation.

**Application dans Pocket Heist :** la spec initiale, plus courte, a été complétée pour aligner `_specs/auth-state-hook.md` sur ce niveau de détail (traduit en français), commit `96f5001` (même commit que ci-dessus), puis implémentée en `94b159e` («feat: ajoute le hook useUser pour l'état d'authentification global », 11/08/2026, 12:55).

## Du spec au plan d'implémentation détaillé, exemple Signup Firebase Integration

**Session source :** 3bb6cf8f, 10/08/2026

**Ce que montre la vidéo :** dix captures VS Code (sous-titres français visibles, donc bien la vidéo) qui montrent la spec `signup-firebase-integration.md` du formateur (connexion du formulaire à `createUserWithEmailAndPassword`, génération d'un «codename» aléatoire en PascalCase comme `displayName`, document Firestore `users` sans stocker l'email), puis son plan d'implémentation détaillé (fonction `generateCodename()`, gestion d'erreurs par code Firebase, tests avec mocks Vitest, vérifications finales).

**Ce que le formateur explique :** comment une spec se traduit en plan d'implémentation concret, étape par étape et vérifiable. Pas juste une liste d'intentions.

**Application dans Pocket Heist :** `_specs/signup-firebase-integration.md` reprend fidèlement la génération de codename et l'exclusion de l'email du document Firestore, commit `1db9f37` («docs: spec pour Signup Firebase Integration », 11/08/2026, 13:03), implémenté ensuite en `d683af4` («feat: connecte le formulaire d'inscription à Firebase Auth », 11/08/2026, 13:10).

## Chronologie complète des specs du projet

Les fonctionnalités suivantes appliquent la même méthode (spec puis plan) sans qu'une nouvelle capture vidéo enseigne une technique différente. Pas de fiche dédiée avec «ce que montre la vidéo» pour chacune, donc. Mais pour savoir précisément ce qui a été fait, et dans quel ordre, voici la suite complète des fichiers de `_specs/`, reconstituée depuis l'historique git réel (`git log --diff-filter=A --follow`) plutôt que depuis les captures.

| # | Fichier                         | Commit                                       | Date                         | Sujet                                                                                                                                                                                 |
|---|---------------------------------|----------------------------------------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | template.md                     | d4c3bc8                                      | 06/08<br>17:00               | Le template lui-même (voir plus haut)                                                                                                                                                 |
| 2 | login-signup-forms.md           | df83e8d                                      | 10/08<br>09:20               | Formulaires login/signup (UI seule)                                                                                                                                                   |
| 3 | auth-state-hook.md              | 96f5001                                      | 11/08<br>12:51               | Hook useUser (voir plus haut)                                                                                                                                                         |
| 4 | signup-firebase-integration.md  | 1db9f37                                      | 11/08<br>13:03               | Connexion Firebase Auth du formulaire d'inscription (voir plus haut)                                                                                                                  |
| 5 | logout-button.md                | 50ddb7d<br>puis<br>renommé<br>par<br>4c185e7 | 11/08<br>14:32<br>→<br>15:29 | Bouton de déconnexion, créé sous le nom logout-functionality.md, renommé le jour même                                                                                                 |
| 6 | login-form.md                   | e8daee6                                      | 11/08<br>16:26               | Formulaire de login (UI seule)                                                                                                                                                        |
| 7 | login-form-functionality.md     | b674c2d                                      | 11/08<br>16:39               | Connexion Firebase Auth du formulaire de login, spec distincte de la précédente, rédigée 13 min après. Même découpage UI/logique que login-signup-forms → signup-firebase-integration |
| 8 | route-protection-auth-guards.md | abf8300                                      | 11/08<br>17:45               | Protection des routes selon l'état d'authentification                                                                                                                                 |
| 9 | create-heist-form.md            | 6b9f2ec                                      | 12/08<br>16:19               | Formulaire de création de heist                                                                                                                                                       |

Ce tableau révèle un pattern répété que les captures vidéo isolées ne montrent pas. Chaque fonctionnalité connectée à Firebase est spécifiée en deux temps, d'abord l'UI seule ( login-signup-forms.md , login-form.md ), puis sa connexion réelle au backend dans un fichier séparé ( signup-firebase-integration.md , login-form-functionality.md ). Ce découpage n'apparaît dans aucune capture retenue plus haut, mais c'est la méthode réellement suivie tout du long.

Un point vaut aussi d'être signalé. Dans la session `ecc9fb7c`, le premier plan proposé par Claude pour le formulaire Create Heist (avec un composant séparé `CreateHeistForm`) a été rejeté, puis entièrement réécrit pour coller à la structure vue dans la vidéo du formateur. Bon exemple concret de l'usage du mode plan comme étape de relecture et de correction avant le code.

**Non couvert par cette reconstitution** : un dixième fichier, `_specs/use-heists-hook.md`, existe dans le dépôt mais n'a jamais été commité (`git status` le montre encore en `??`, non suivi). Travail en cours, postérieur à ce qui a été analysé ici, à documenter séparément une fois committé.

## Section 4 : MCP Servers

---

*Cours : Claude Code Masterclass, Section 4 (13/13 leçons, 1h40).*

Cette section couvre un seul serveur MCP réellement documenté par une capture de la vidéo dans les 8 sessions analysées, Firebase (plan Firestore et Authentication). Figma MCP et Context7 sont bien utilisés dans le projet, mais aucune image collée ne montre le formateur en train de les expliquer. Voir la note en fin de section.

### Le plan Firebase MCP : Firestore + Authentication en mode test

**Sessions source :** 8d281e45 (10/08 matin) et 3bb6cf8f (10/08, suite, la même capture a été collée dans les deux sessions)

**Ce que montre la vidéo :** un terminal Claude Code appartenant au formateur (chemin visible en bas de l'image, `/Users/shaun/Code/Courses/pocket_heist`, à distinguer de mon environnement WSL). Le serveur Firebase MCP vient de proposer un plan en 5 étapes après la commande d'initialisation.

1. Install Firebase SDK, ajouter Firebase à `package.json` et créer le fichier de config SDK.
2. Initialize Firestore, règles de sécurité en mode test, accès ouvert pour le développement.
3. Deploy Firestore, `firebase deploy --only firestore`.
4. Enable Authentication, activer l'authentification Email/Password dans la console Firebase.
5. Implement Auth in App, ajouter l'initialisation du SDK Auth dans le code.

L'image montre ensuite l'appel d'outil MCP concret qui suit l'acceptation du plan, l'outil `firebase` invoqué pour «Get Firebase SDK Config », avec le prompt de permission standard («Do you want to proceed? Yes / Yes and don't ask again / No »).

**Ce que le formateur explique :** à quoi ressemble un plan généré automatiquement par un serveur MCP puissant comme Firebase MCP, capable de créer de la vraie infrastructure cloud (base de données, authentification), et pourquoi rester minimaliste dans la demande initiale, mode test uniquement, pas de hosting. Un point important est signalé en comparant avec mon environnement, le plugin Firebase MCP a changé de comportement par défaut depuis l'enregistrement de la vidéo, règles moins ouvertes par défaut aujourd'hui. Il fallait donc suivre explicitement la version du formateur pour rester en mode test ouvert.

**Application dans Pocket Heist :** j'ai suivi ce plan à l'identique dans mon propre terminal. Installation du SDK Firebase (`npm install firebase`), écriture de `firestore.rules` en mode test ouvert (`allow read, write: if request.time < timestamp.date(2026, 9, 10);`), `firebase.json`, déploiement (`firebase deploy --only firestore`), activation de l'authentification Email/Password, puis création de `lib/firebase/config.ts`. Commit `083b5a6` («feat: configure Firebase (Firestore + Authentication) », 2026-08-11 10:44).

---

## Note sur Figma MCP et Context7

Le projet utilise bien ces deux autres serveurs MCP concrètement.

- **Figma MCP**, commit `f336a1f` («feat: style le bouton Create Heist selon le design Figma », 2026-08-11 01:08), et un tag `figma_component (if used)` ajouté au template de spec, commit `395e229` («feat: ajoute la référence Figma optionnelle à la commande /spec », 2026-08-11 01:08).
- **Context7**, la consigne de vérifier la doc via Context7 avant d'écrire du code a été ajoutée directement dans `CLAUDE.md`, commit `dadbd08` («docs: ajoute la consigne de vérification doc via Context7 dans CLAUDE.md », 2026-08-10 12:52).

Mais en vérifiant chaque image des 8 sessions, notamment l'image Figma de la session `8d281e45/3bb6cf8f`, qui montre en fait mon fichier Figma personnel avec le bandeau «View only », pas une capture de la vidéo, aucune n'affiche le formateur en train d'expliquer Figma MCP ou Context7 à l'écran. Ces deux usages ont donc été appliqués sans capture vidéo correspondante dans le lot d'images fourni. Pas de leçon sourcée pour eux, pour ne pas inventer un contenu qui n'a pas été vérifié.

## Section 5 : Plugins & Skills

---

Cours : Claude Code Masterclass, Section 5 (6/7 leçons, 46 min).

Cette section du cours n'a laissé aucune trace directe dans les 8 sessions analysées. Aucune des images collées ne montre le formateur en train d'expliquer la notion de «Skill » ou de «Plugin » Claude Code (créer un fichier `SKILL.md` , installer un plugin, parcourir un marketplace, etc.).

### Aucune leçon vidéo identifiée pour cette section

Après lecture des 8 rapports et vérification visuelle des images candidates, rien ne correspond à la Section 5 telle que décrite dans la table des matières Udemy.

#### Ce qui a été vérifié et écarté :

- **Le skill** `.claude/skills/firestore-schemas/Skill.md` , créé par Claude lui-même dans le commit `241a4cb` («update heist type », 12/08 12:26) pendant la génération des types Firestore, puis étendu dans `6b9f2ec` («feat: formulaire Create Heist », 12/08 16:19) après la découverte d'un bug de typage ( `FirestoreDataConverter` attendait `WithFieldValue<X>` et non `Partial<X>` ), documenté après coup pour éviter que le bug se reproduise. Bonne pratique appliquée par Claude, mais aucune image collée ne montre le formateur créer ou expliquer ce mécanisme de skill. L'image de la session `62cf38a7` (et sa quasi-copie dans `ecc9fb7c`) montre juste le texte d'un prompt sur les champs du document Firestore `heist` , rien sur la notion de skill elle-même. Cette leçon appartient plutôt à la [Section 4 \(MCP Servers / Firebase\)](#), ou reste une pratique interne à Claude, pas un contenu de la Section 5.
- **Les commandes** `/component` et `/commit-message` , présentes dans `.claude/commands/` (créées le 06/08). Le récapitulatif de la session `8d281e45` les classe lui-même, via le sommaire réel du cours affiché à l'écran, sous «Section 2 : Commands, Context, Tools & Hooks ». Pas sous Plugins & Skills. Ce sont des commandes personnalisées (custom slash commands), un mécanisme distinct des skills. Voir [section-2-commands-context-tools-hooks.md](#) .
- **La commande** `/spec` (commit `d4c3bc8` ), même chose. Aucune image des 8 sessions ne montre sa création depuis la vidéo. Les seules leçons vidéo captées à son sujet portent sur le contenu des specs (template, sections, formulation), qui relève de la [Section 3 \(Plan Mode & Specs\)](#), déjà couverte ailleurs.
- **Les hooks** `PostToolUse` (sessions `bc2538f9` et `a056c64e`), bien documentés, mais explicitement Section 2, pas Plugins & Skills.

## Conclusion

Aucune leçon confirmée pour cette section. Les seuls objets du projet qui portent le nom «skill » ( `.claude/skills/firestore-schemas/` , `.claude/skills/humanizer/` ) ou qui ressemblent à des plugins ne sont pas rattachables à une capture vidéo. Soit ce sont des initiatives autonomes de Claude, soit ils appartiennent aux Sections 2, 3 ou 4 déjà traitées ailleurs dans ce dossier. Si le formateur a bien consacré 46 minutes à ce sujet dans le cours, je ne l'ai pas encore suivi, ou pas collé de capture, au moment des 8 sessions couvertes ici.